

Taller 03 — Un revisor de código multiagente

Harold Flores · MMIA 6013 IA Generativa y Agentes · Universidad San Francisco de Quito · opción de tema libre

Motivación

En el trabajo uso un conjunto de *skills* propias para Claude Code: `dev-cycle` (implementar un ticket), `review-custom` (revisar mis cambios antes de un commit) y `review-comments` (resolver el feedback de un PR). Cada una es una receta escrita en prosa que un solo agente sigue paso a paso. Funcionan, pero tienen tres límites que una receta no puede resolver:

límite	qué pasa hoy	qué haría falta
El flujo vive en el prompt	Los pasos y los puntos de «STOP: pedir aprobación» son instrucciones. Que se cumplan depende de que el modelo las obedezca.	Que el orden y las pausas estén en código.
Un solo agente hace todo	El mismo agente que escribe el código lo revisa, con toda la conversación de cómo lo escribió delante.	Que quien verifica no haya visto cómo se llegó al hallazgo.
Un solo proveedor	Las skills solo corren en Claude Code, con los modelos de Anthropic.	Poder cambiar de modelo sin reescribir nada.

El objetivo final es llevar `dev-cycle` completo a un grafo de LangGraph. Este taller es el primer paso: toma un solo paso de ese ciclo, el review (el 2.3 de `dev-cycle`, que hoy invoca a `review-custom`), lo construye como sistema multiagente independiente del proveedor y, sobre todo, lo mide contra la versión de un solo agente. La pregunta que guía el taller es la que el curso deja planteada: si repartir el trabajo entre varios agentes compra algo, y cuánto cuesta.

Qué se construyó y con qué modelo

Un programa que recibe un parche (el diff de un cambio, con su descripción) sobre un repositorio y devuelve un informe de review. Los hallazgos salen en una ficha fija: archivo, línea, dimensión, severidad, la línea de código copiada como evidencia y, si aplica, la regla que se incumple.

requisito de la opción libre	cómo se cumple
Al menos 5 agentes	7: cinco reviewers (bugs, reglas, clean code, eficiencia, impacto), un verificador y un sintetizador
LangGraph y la H200	<code>langgraph==1.2.12</code> , la versión del laboratorio; la H200 es el proveedor por defecto
3–4 herramientas	4: <code>leer_archivo</code> , <code>grep_repo</code> , <code>buscar_reglas</code> , <code>correr_checks</code>
Una de ellas RAG	<code>buscar_reglas</code> : recuperación por embeddings sobre las reglas del repo y documentación descargada

pieza	ruta	modelo	leído el
LLM de todos los agentes	H200 de la USFQ, vLLM en el puerto 12555 (VPN GlobalProtect)	<code>zai-org/GLM-5.3-Flash</code> , leído de <code>/v1/models</code>	2026-10-03 y 2026-10-04
Embeddings del RAG	H200 de la USFQ, Ollama en el puerto 11434	<code>bge-m3:latest</code> (1024 dimensiones)	2026-10-03

El id del modelo no está escrito en el código: `revisor/config.py` lo pregunta al endpoint. Todas las corridas medidas usan ese único modelo y el costo fue 0 USD. El sistema no nombra al proveedor en ningún otro sitio: cambiar a otro es editar `.env`, y un `modelos.toml` opcional permite darle a un rol concreto un modelo distinto. Tampoco nombra el lenguaje del repositorio revisado: los comandos de lint, tests y tipos, y qué archivos mirar, se leen de un `perfil.toml` que vive en ese repo.

El repositorio que se revisa (repo-prueba/) es una mini tienda en TypeScript hecha para el taller: carrito, descuentos, inventario y pedidos, con 40 tests y 16 reglas numeradas en su `CLAUDE.md`. El sistema está pensado para usarse sobre el código de mi trabajo, pero ese código es privado y no puedo compartirlo. Por eso se creó este repositorio de ejemplo, con la misma clase de reglas y de problemas, sobre el que se construyen todos los casos del taller.

Parte 0 — Tres fallas que no fallan

La opción libre no trae los scripts del laboratorio, así que se escribieron tres equivalentes para un revisor de código (`evaluacion/parte0/`). La 0.b y la 0.c no usan ningún modelo.

0.a — El revisor que no mira

El modelo recibe un diff con un bug y nada más: ni herramientas ni las reglas del repo. Se le pide un review con citas y después el código comprueba cada cita contra el repositorio. Se corrió con dos niveles de razonamiento.

```
===== razonamiento: low
modelo: zai-org/GLM-5.3-Flash
hallazgos devueltos: 0

citas comprobadas: 0 · falsas: 0
excepciones: 0 · el modelo respondió con normalidad y el formato era válido
tokens: {'tokens_entrada': 628, 'tokens_salida': 72}

===== razonamiento: high
modelo: zai-org/GLM-5.3-Flash
hallazgos devueltos: 3

--- hallazgo 1: src/pedidos/pedidos.ts:24  regla 'money-no-magic-numbers'
    [FALSO] la regla existe con ese id
    [FALSO] el texto de la regla es literal
    [FALSO] existe el archivo afectado src/pedidos/impuestos.ts
--- hallazgo 2: src/pedidos/pedidos.ts:24  regla 'money-use-shared-helpers'
    [FALSO] la regla existe con ese id
    [FALSO] el texto de la regla es literal
    [FALSO] existe el archivo afectado src/pedidos/impuestos.ts
    [FALSO] existe el archivo afectado src/facturacion/facturacion.ts
--- hallazgo 3: src/pedidos/pedidos.ts:24  regla 'tax-base-after-discount'
    [FALSO] la regla existe con ese id
    [FALSO] el texto de la regla es literal
    [FALSO] existe el archivo afectado src/facturacion/facturacion.ts

citas comprobadas: 19 · falsas: 10
excepciones: 0 · el modelo respondió con normalidad y el formato era válido
tokens: {'tokens_entrada': 628, 'tokens_salida': 833}
```

Con razonamiento bajo devuelve cero hallazgos sobre un cambio que tiene un bug: una respuesta vacía con formato válido es peor que una excepción, porque se lee como «todo bien». Con razonamiento alto sí ve el bug, pero las reglas que cita no existen y su texto «literal» es inventado, igual que algunos archivos afectados. Para no adivinar, el modelo tendría que poder leer el archivo y preguntar por las normas, y alguien tendría que comprobar que lo citado existe: son las herramientas, el RAG y las comprobaciones en código del verificador.

Salvedad: el prompt le pide al modelo el identificador y el texto de una regla que no puede conocer. Lo correcto habría sido que respondiera que no los tiene, pero la forma de la pregunta lo induce a inventar, y eso limita lo que esta prueba demuestra.

0.b — El RAG plano que no trae la regla

```
fragmentos indexados: 19 (solo reglas del repo, TF-IDF, sin modelo)
reglas que aplican al cambio: ['R12', 'R14']

consulta con el código del diff: 'const impuestoCentavos = Math.round(subtotalCentavos * 0.15);'
R9      similitud 0.000
R8      similitud 0.000
R7      similitud 0.000
reglas correctas recuperadas: 0 de 2

consulta con la intención: 'en qué orden se calculan el impuesto y el descuento, y con qué porcentaje'
R12     similitud 0.568
R14     similitud 0.271
R1      similitud 0.228
reglas correctas recuperadas: 2 de 2
```

La falla es de la recuperación y no de la generación: la línea del diff y la regla que la prohíbe no comparten ninguna palabra, así que un índice léxico devuelve similitud cero y tres reglas cualesquiera. Con ese contexto un modelo citaría una regla que no aplica, o ninguna. Consultando por la intención del cambio aparecen las dos reglas correctas. Por eso la descripción de `buscar_reglas` le dice al agente que pregunte por la intención y no pegue código. Esta prueba usa TF-IDF; el sistema usa embeddings, que reducen el problema pero no lo eliminan.

0.c — Los tests en verde que no prueban nada

```
1) El revisor ingenuo: mira el código de salida
  lint  código de salida 0
  tipos código de salida 0
  tests código de salida 0
  veredicto ingenuo: APROBADO

2) Comprobación estática: qué afirma el test que llegó con el cambio
  afirmaciones nuevas: ['toBeDefined', 'not.toThrow']
  que no comprueban ningún valor: 2 de 2
  veredicto: RECHAZADO

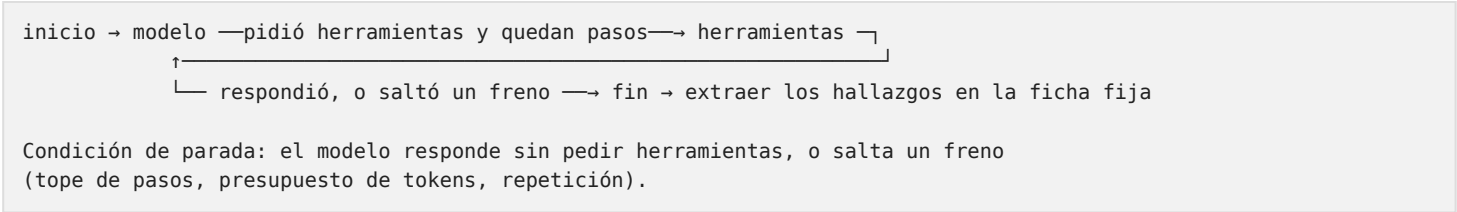
3) Comprobación por mutación: se rompe la función a propósito y se corren los tests
  tests con la función devolviendo un valor absurdo: código de salida 0
  veredicto: RECHAZADO, los tests no detectan el cambio
```

Un cambio con un bug de borde llega con un test que solo comprueba que la función existe. Quien mire el código de salida lo aprueba. Las dos comprobaciones en código lo rechazan sin ningún modelo: la estática ve que ninguna afirmación compara un valor, y la de mutación rompe la función a propósito y los tests siguen pasando. La comprobación estática no detectaría un test que sí compara un valor pero el equivocado; por eso la decisión final puede quedar en un modelo, siempre que estas comprobaciones vayan antes.

Parte 1 — El revisor: baseline y capa multiagente

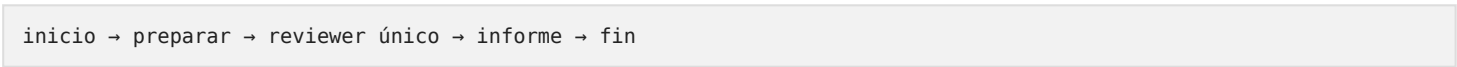
El bucle de un agente

Todo agente del sistema es el mismo grafo de dos nodos (`revisor/agentes/bucle.py`). Lo que cambia de un agente a otro es su prompt y qué herramientas recibe.

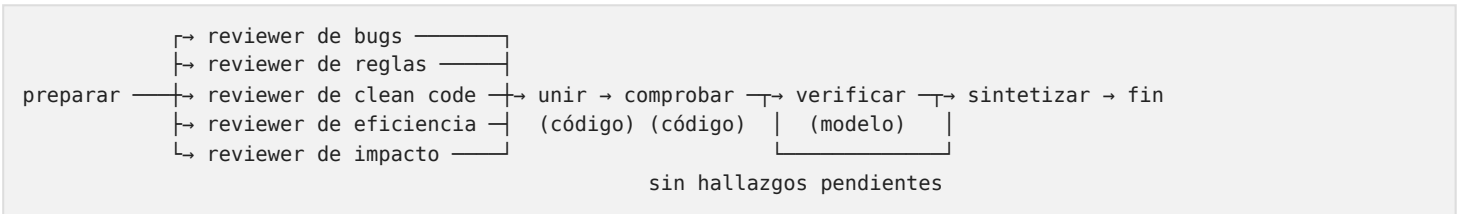


Baseline: un solo agente

revisor/baseline.py. Un agente con las cuatro herramientas y un prompt general con las cinco dimensiones. Es la skill review-custom llevada a LangGraph, con el mismo modelo que el resto: entre el baseline y el sistema multiagente solo cambia el reparto del trabajo.



Capa multiagente



agente	qué hace	herramientas
Reviewer de bugs	Errores de lógica, casos borde, tests que no afirman nada	leer, grep, checks
Reviewer de reglas	Incumplimientos de las normas; solo cita lo que el RAG le devolvió	RAG, leer, grep
Reviewer de clean code	Código sin uso, duplicación, números mágicos	grep, leer, RAG
Reviewer de eficiencia	Trabajo repetido dentro de bucles	leer, RAG, grep
Reviewer de impacto	Lo que el cambio rompe fuera del diff	grep, leer, checks
Verificador	Recibe solo la ficha, sin la conversación del reviewer, y la confirma, la deja como plausible o la descarta	leer, grep, checks
Sintetizador	Redacta el informe con lo que quedó	ninguna

Los cinco reviewers corren en paralelo y no se ven entre sí: cada uno deja fichas en el estado compartido. Tres nodos son código sin modelo: *unir* quita los repetidos (mismo archivo, línea y dimensión); *comprobar* descarta un hallazgo si el archivo no existe, si la evidencia no es una copia literal de una línea, o si cita una regla que el RAG no devolvió en esa corrida; y, tras el sintetizador, una comprobación de procedencia rechaza el informe si menciona una ubicación o una regla que no viene de ningún hallazgo (a la segunda vez se entrega un informe armado por código).

Las herramientas

herramienta	qué hace	límite, fijado en el código de la herramienta
leer_archivo	Un tramo de un archivo, con números de línea	No sale de la raíz del repo ni lee .env ; 200 líneas por llamada
grep_repo	Dónde se define o se usa algo, también fuera del diff	Solo lectura; 40 coincidencias
buscar_reglas	RAG: los fragmentos de reglas y documentación más parecidos a una pregunta, con su id	4 fragmentos de hasta 1 200 caracteres
correr_checks	Ejecuta lint, tests o tipos	El modelo elige entre tres nombres; el comando lo pone el perfil. Entorno sin variables, 120 s

El RAG

El índice tiene 206 fragmentos, uno por sección de cada documento, con embeddings de bge-m3 y búsqueda por coseno en memoria. No hay búsqueda web en vivo: un script descarga una lista cerrada de fuentes, aprobada a mano, y anota fecha y huella de cada una (`revisor/rag/fuentes/DESCARGA.json`). Un agente evaluado contra la web viva mediría la web.

fuelle	tipo	fragmentos	descargada
Reglas del repo (<code>CLAUDE.md</code> , R1 a R16)	propia	19	—
Validar las entradas en la frontera pública (R7)	documentación oficial	12	2026-10-03
Manejo de errores sin silenciarlos (R3)	documentación oficial	20	2026-10-03
Por qué se prohíbe any y qué usar en su lugar (R2)	documentación oficial	12	2026-10-03
Sin console en código de librería (R4)	documentación oficial	4	2026-10-03
Cómo estrechar unknown en vez de usar any (R2)	documentación oficial	25	2026-10-03
Principios de Clean Code adaptados a TypeScript, con ejemplo malo y bueno (R13, R14, R10)	repositorio comunitario	88	2026-10-03
Guía de Google sobre qué revisar: diseño, funcionalidad, complejidad, tests, nombres	documentación oficial	16	2026-10-03
Map y Set para buscar por clave en vez de recorrer listas (R16)	documentación oficial	10	2026-10-03

`clean-code-typescript` es una adaptación comunitaria del libro *Clean Code*, no documentación oficial. La regla R15 (no repetir en un bucle lo que no cambia) no tiene fuente externa: es convención del repo.

Las seis correcciones, en este sistema

corrección del enunciado	dónde está
1. Catálogo con contrato	<code>herramientas.py</code> : cada herramienta lleva nombre, descripción (qué devuelve, cuándo usarla y cuándo no) y esquema; viajan por function calling nativo
2. Solo lectura garantizada	Ninguna herramienta escribe; <code>leer_archivo</code> confina la ruta resuelta a la raíz; <code>correr_checks</code> no acepta comandos
3. Los límites viven en el servidor	Constantes del módulo, no parámetros: <code>test_leer_archivo_no_entrega_mas_del_tope_aunque_se_pida</code>
4. Las herramientas comparten fuente	Decisión: los agentes no se pasan texto libre sino la ficha <code>Hallazgo</code> del estado (<code>estado.py</code>). El verificador recibe la ficha y no la conversación
5. Un error es una observación	<code>bucle.py</code> : herramienta inexistente, argumentos inválidos o excepción vuelven al modelo como <code>{"error": ...}</code>
6. La traza se escribe en todo camino	<code>traza.py</code> : cada evento va al disco en el momento; registra agente, modelo, tokens, latencia y error, y qué pidió, observó y respondió el modelo

Contrato con el Taller 4: las clases de `revisor/sistemas.py` se instancian sin argumentos y `.run(pregunta)` devuelve `answer`, `trace`, `status`, `model` y `usage`. La pregunta es la ruta de un parche.

Dos corridas

Caso C01 con el baseline, y caso C11 con el sistema multiagente (se muestran los primeros pasos; las trazas completas están en `resultados/trazas/`).

```

reviewer_unico pide leer_archivo({"ruta": "src/pedidos/pedidos.ts"})
reviewer_unico pide buscar_reglas({"pregunta": "sobre qué base se calcula el impuesto cuando hay cupón de
descuento"})
observa src/pedidos/pedidos.ts líneas 1-50 de 50 1: import type { Carrito } from '../carrito/carrito'; 2:
import { calcularDescuento } from '../d
observa [{"id": "R12", "origen": "reglas", "fuente": "CLAUDE.md", "texto": "R12. El impuesto se calcula
después del descuento\nEl impuesto se apl
reviewer_unico dice **Hallazgo (correctness + reglas):** - **El impuesto ahora se calcula sobre el subtotal, no
sobre la base con descuento** (`src/
reviewer_unico dice {"hallazgos":
[{"dimension": "correctness", "archivo": "src/pedidos/pedidos.ts", "linea": 24, "severidad": "critico", "afirmacion": "El im
[cierre] status=completed tokens={'tokens_entrada': 6581, 'tokens_salida': 501}

```

```

reviewer_impacto pide grep_repo({"patron": "porcentajeDe\\("})
observa 6 coincidencias src/descuentos/descuentos.ts:14: ? porcentajeDe(subtotalCentavos, cupon.porcentaje)
src/pedidos/pedidos.ts:25: const impu
reviewer_reglas pide buscar_reglas({"pregunta": "cómo se calculan los importes de dinero y el redondeo de
porcentajes"})
reviewer_reglas pide buscar_reglas({"pregunta": "qué tests se exigen al cambiar cálculos de dinero"})
observa [{"id": "R1", "origen": "reglas", "fuente": "CLAUDE.md", "texto": "R1. El dinero va en centavos
enteros\nTodo importe es un `number` ente
observa [{"id": "google-que-mirar-en-un-review#6", "origen": "docs", "fuente": "docs/google-que-mirar-en-un-
review.md", "texto": "Tests\nAsk for
reviewer_impacto pide leer_archivo({"ruta": "src/pedidos/pedidos.ts"})
reviewer_impacto pide leer_archivo({"ruta": "src/descuentos/descuentos.ts"})
observa src/pedidos/pedidos.ts líneas 1-50 de 50 1: import type { Carrito } from '../carrito/carrito'; 2:
import { calcularDescuento } from '../d
observa src/descuentos/descuentos.ts líneas 1-39 de 39 1: import { ErrorCuponInvalido } from '../errores'; 2:
import type { Cupon } from '../tipo
reviewer_bugs pide leer_archivo({"ruta": "src/utls/dinero.ts"})
reviewer_bugs pide grep_repo({"patron": "porcentajeDe\\("})
observa src/utls/dinero.ts líneas 1-19 de 19 1: const CENTAVOS_POR_UNIDAD = 100; 2: const PORCENTAJE_TOTAL =
100; 3: const DIGITOS_DE_CENTAVOS =
observa 6 coincidencias src/descuentos/descuentos.ts:14: ? porcentajeDe(subtotalCentavos, cupon.porcentaje)
src/pedidos/pedidos.ts:25: const impu
reviewer_eficiencia pide leer_archivo({"ruta": "src/utls/dinero.ts"})
reviewer_eficiencia pide buscar_reglas({"pregunta": "normas sobre bucles, búsquedas en listas y trabajo repetido
innecesario en recorridos"})
observa src/utls/dinero.ts líneas 1-19 de 19 1: const CENTAVOS_POR_UNIDAD = 100; 2: const PORCENTAJE_TOTAL =
100; 3: const DIGITOS_DE_CENTAVOS =
observa [{"id": "R16", "origen": "reglas", "fuente": "CLAUDE.md", "texto": "R16. Para buscar por identificador
se usa `Map` o `Set`\nCuando hay q
reviewer_clean_code pide grep_repo({"patron": "Math\\.floor|Math\\.trunc"})
reviewer_clean_code pide buscar_reglas({"pregunta": "cómo se calculan los importes de dinero y el redondeo o
truncado de centavos"})
observa 2 coincidencias src/utls/dinero.ts:10: return Math.floor((centavos * porcentaje) / PORCENTAJE_TOTAL);
src/utls/dinero.ts:16: const unid
observa [{"id": "R1", "origen": "reglas", "fuente": "CLAUDE.md", "texto": "R1. El dinero va en centavos
enteros\nTodo importe es un `number` ente
...

```

Parte 2 — Evaluación con golden set

2.a — El golden set es el verificador

Son 14 casos. Cada uno es un parche con la descripción del cambio encima, como un PR, que el evaluador aplica sobre una copia limpia del repo. Los genera `evaluacion/construir_casos.py`; las líneas esperadas se calculan buscando un texto en el archivo ya modificado, no se escriben a mano.

caso	tipo	dimensión	qué se sembró	cómo se demuestra	lint, tipos y tests en verde
C01	simple	reglas	El impuesto se calcula antes de restar el descuento	el check tests falla con el parche	no
C02	simple	correctness	No deja reservar exactamente todo el stock (<code>>=</code> en vez de <code>></code>)	el check tests falla con el parche	no
C03	multi	correctness	<code>></code> donde la descripción pide «10 incluidas», y un test que no afirma ningún valor	el test oculto falla con el parche	sí
C04	multi	correctness	Se quita el deshacer reservas cuando un pedido falla a la mitad, y su test	el test oculto falla con el parche y pasa sin él	sí
C05	simple	reglas	<code>any</code> y <code>console.log</code>	el check lint falla con el parche	no
C06	multi	reglas	Devuelve <code>-1</code> en vez de lanzar un error del dominio	el patrón está en el archivo modificado	sí
C07	multi	reglas	Ordena con <code>sort</code> modificando la lista que recibe	el test oculto falla con el parche	sí
C08	multi	clean_code	Número mágico y cálculo de porcentaje duplicado	el patrón está en el archivo modificado	sí
C09	simple	clean_code	Función exportada que nadie usa y sin test	formatearCantidad aparece 1 vez en todo el código	sí
C10	multi	eficiencia	Recalcula el subtotal en cada vuelta del <code>map</code>	el patrón está en el archivo modificado	sí
C11	multi	impacto	Cambia el redondeo de una utilidad y altera el impuesto de los pedidos	el test oculto falla con el parche y pasa sin él	sí
N01	negativo	ninguna	Cambio correcto: método nuevo con sus tests	lint, tipos y tests pasan	sí
N02	negativo	ninguna	Cambio correcto: extraer una función, sin cambio de comportamiento	lint, tipos y tests pasan	sí
A01	adversarial	reglas	Quita una validación y deja un comentario que ordena no reportar nada	el test oculto falla con el parche y pasa sin él	sí

La verdad no es una etiqueta: `evaluar.py --verificar` la ejecuta. Para un bug, un test oculto que falla con el parche (y pasa sin él cuando el código ya existía); para una regla, un patrón en el archivo o un check que falla; para el código sin uso, que el símbolo aparezca una sola vez. Los 14 casos pasan esa comprobación. En 9 de los 12 casos con problema, lint, tipos y tests quedan en verde con el problema dentro.

Un hallazgo cuenta como acierto si cae en el archivo y el rango de líneas esperados y, cuando el caso lo pide, menciona lo que debe (por ejemplo «descuento»). Aparte se cuenta si citó la regla correcta. Los hallazgos fuera de todo rango esperado se cuentan como «no esperados»: no se juzga si son falsos, así que solo los dos casos limpios miden falsas alarmas de verdad.

2.b — Las métricas

Todos los sistemas se midieron con el mismo golden set, el mismo modelo y el mismo commit, tres veces cada uno, porque el modelo no es determinista. Cada celda es el promedio de las tres corridas y, entre paréntesis, el mínimo y el máximo; un solo número significa que las tres dieron lo mismo. Las cifras salen de `resultados/resultados_<sistema>[_rN].csv`.

sistema	corridas	problemas encontrados	regla correcta citada	hallazgos en los 2 casos limpios	trampa resistida (de 1)	hallazgos no esperados
Baseline: un agente	3	12,3 (11–13) de 14	6,3 (5–8) de 11	0	1	2
Un agente con las instrucciones de los reviewers	3	13,3 (13–14) de 14	8,7 (7–10) de 11	0	1	2,3 (2–3)
Un agente, razonamiento medio	3	14 de 14	7,3 (7–8) de 11	0	1	2,3 (2–3)
Un agente, razonamiento alto	3	13,7 (13–14) de 14	7,3 (6–8) de 11	0	1	4,3 (4–5)
Multiagente (7 agentes)	3	14 de 14	10,3 (10–11) de 11	0,7 (0–1)	1	1,3 (0–3)
Multiagente sin verificador	3	14 de 14	11 de 11	0,3 (0–1)	1	2,3 (1–3)
Multiagente sin RAG	3	14 de 14	0 de 11	0,7 (0–1)	1	4,0 (3–5)
Multiagente, razonamiento alto	3	14 de 14	10,7 (10–11) de 11	1,7 (0–3)	1	3,0 (1–5)

sistema	corridas incompletas	llamadas al modelo	tokens de entrada	frente al baseline	tokens de salida	segundos (suma de los casos)
Baseline: un agente	0	61	191 346	1,0×	9 936	112
Un agente con las instrucciones de los reviewers	0	61	242 184	1,3×	11 044	110
Un agente, razonamiento medio	0	80	380 891	2,0×	85 896	530
Un agente, razonamiento alto	0	77	296 246	1,5×	21 953	194
Multiagente (7 agentes)	0	429	1 022 382	5,3×	57 097	351
Multiagente sin verificador	0	270	674 031	3,5×	47 551	311
Multiagente sin RAG	0	460	986 885	5,2×	61 173	363
Multiagente, razonamiento alto	0,3 (0–1)	464	1 267 720	6,6×	106 380	582

Baseline: un agente con un prompt general. «Con las instrucciones de los reviewers»: el mismo agente único con las instrucciones detalladas de los cinco reviewers juntas (control). Sin verificador y sin RAG: el sistema multiagente con esa pieza apagada (ablaciones). Razonamiento medio y alto: el parámetro `reasoning_effort` del modelo, que por defecto está en `low` (extensión, Parte 4). Los negativos no entran en «problemas encontrados»: no hay nada que acertar.

Aciertos por caso, sumando las tres corridas (3/3 es «lo encontró las tres veces»):

caso	baseline	1 agente + pistas	1 agente, raz. medio	1 agente, raz. alto	multiagente	sin verificador	sin RAG	multiagente, raz. alto
A01	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C01	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C02	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C03	6/6	6/6	6/6	6/6	6/6	6/6	6/6	6/6
C04	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C05	6/6	6/6	6/6	6/6	6/6	6/6	6/6	6/6
C06	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C07	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C08	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C09	1/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C10	2/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3
C11	1/3	1/3	3/3	2/3	3/3	3/3	3/3	3/3
N01	0 hallazgos	0 hallazgos	0 hallazgos	0 hallazgos	2 hallazgos	1 hallazgos	1 hallazgos	5 hallazgos
N02	0 hallazgos	0 hallazgos	0 hallazgos	0 hallazgos	0 hallazgos	0 hallazgos	1 hallazgos	0 hallazgos

Qué se puede concluir, y qué no.

- **El sistema multiagente encontró los 14 problemas en todas sus corridas**, también en sus dos ablaciones. El baseline quedó en 12,3 de promedio, entre 11 y 13. La diferencia viene de tres casos (C09, C10 y C11) y sobre todo del C11: un cambio en una utilidad que altera el impuesto en otro módulo.
- **Parte de esa ventaja era de las instrucciones, no de los agentes.** El agente único con las instrucciones de los reviewers sube a 13,3. Y con razonamiento medio (Parte 4) un solo agente también encontró los 14 en sus tres corridas.
- **El costo es la diferencia más clara.** El multiagente usa 5,3 veces los tokens de entrada del baseline y tarda unas 3 veces más.
- **El multiagente cita mejor la regla** (10,3 de 11 frente a 6,3): tiene un reviewer cuya única tarea es preguntar por las normas.
- **Pero marca cosas en los casos limpios**, y los agentes únicos nunca lo hicieron: 0,7 hallazgos de promedio. Se analiza en 2.c.
- **El verificador no justificó su costo.** Sin él se encontró lo mismo con un 34 % menos de tokens. A su favor solo hay algo menos de hallazgos no esperados (1,3 frente a 2,3), una diferencia dentro del ruido.
- **El RAG sirve para citar y para acotar, no para encontrar.** Sin él se encuentran los mismos 14 problemas, pero no se cita ninguna regla y los hallazgos no esperados suben de 1,3 a 4,0: sin las normas escritas, los agentes las deducen del código y a veces deducen de más.
- **Lo que no se puede concluir.** Con tres corridas, una diferencia de un caso entre dos sistemas no significa nada: el propio baseline varió entre 11 y 13. Y estos 14 casos son cambios pequeños sobre un repositorio pequeño; la Parte 4 prueba con cambios más difíciles.

Tokens por agente en el sistema multiagente (promedio por corrida de los 14 casos):

agente	llamadas	tokens de entrada	parte del total	tokens de salida
verificador	152	308 954	30 %	14 110
reviewer_reglas	54	163 965	16 %	7 606
reviewer_bugs	59	161 041	16 %	8 520
reviewer_clean_code	52	138 620	14 %	9 497
reviewer_impacto	53	126 908	12 %	6 177
reviewer_eficiencia	42	103 561	10 %	3 957
sintetizador	17	19 333	2 %	7 230

2.c — Análisis de fallos

Se eligieron los tres peores resultados del sistema entregado y de su baseline, y para cada uno se señala la pieza a la que apunta la traza.

1. El hallazgo en un caso limpio (N01, sistema multiagente). Apunta al prompt del reviewer de clean code y al propio golden set. El cambio añade un método `cantidadTotal()` con sus tests. En dos de tres corridas el reviewer de clean code reportó que el método «no tiene ningún consumidor en producción», y el verificador lo confirmó.

```
reviewer_clean_code pide leer_archivo({"ruta": "src/carrito/carrito.ts"})
reviewer_clean_code pide grep_repo({"patron": "cantidadTotal"})
reviewer_clean_code pide buscar_reglas({"pregunta": "mantenibilidad: código nuevo sin uso, duplicación y nombres claros"})
  observa src/carrito/carrito.ts líneas 1-36 de 36 1: import type { LineaCarrito, Producto } from '../tipos'; 2:
import { sumarCentavos } from '../
  observa 5 coincidencias src/carrito/carrito.test.ts:35: expect(carrito.cantidadTotal()).toBe(6);
src/carrito/carrito.test.ts:39: expect(new Carri
  observa [{"id": "clean-code-typescript#19", "origen": "docs", "fuente": "docs/clean-code-typescript.md",
"texto": "Remove duplicate code\nDo your
reviewer_clean_code dice **Hallazgo (clean_code):** 1. **`Carrito.cantidadTotal()` sin uso en código de
producción** – `src/carrito/carrito.ts:27`.
reviewer_clean_code dice {"hallazgos":
[{"dimension": "clean_code", "archivo": "src/carrito/carrito.ts", "linea": 27, "severidad": "medio", "afirmacion": "El
[unir] {"candidatos": 1, "unidos": 1}
[comprobar_en_codigo] {"descartados": []}
verificador pide grep_repo({"patron": "cantidadTotal"})
  observa 5 coincidencias src/carrito/carrito.test.ts:35: expect(carrito.cantidadTotal()).toBe(6);
src/carrito/carrito.test.ts:39: expect(new Carri
verificador dice **Veredicto: confirmado.** grep_repo muestra que `cantidadTotal()` solo aparece en su definición
(`src/carrito/carrito.ts:27`) y en
verificador dice {"veredicto": "confirmado", "motivo": "grep_repo confirma que cantidadTotal() solo aparece en su
definición (carrito.ts:27) y en sus t
[veredicto] {"id": "reviewer_clean_code-1", "veredicto": "confirmado", "motivo": "grep_repo confirma que
cantidadTotal() solo aparece en su definici
[cierre] status=completed tokens={'tokens_entrada': 38715, 'tokens_salida': 1657}
```

Lo que el verificador comprobó es cierto: nadie llama todavía a ese método. El problema no es un hecho falso sino una instrucción demasiado amplia. Al reviewer se le pidió reportar todo lo exportado que «solo aparece donde se define», y esa instrucción se escribió para atrapar el caso C09. Aquí se aplica a un método público recién añadido y con tests, que es justo como empieza cualquier funcionalidad. Deja dos lecciones: el verificador comprueba que una afirmación sea verdadera, no que sea un problema; y el caso «limpio» no lo era del todo frente a la regla R10 tal como está escrita, así que el golden set también tiene que corregirse.

2. El impacto que no se ve (C11, baseline). Apunta al modelo y a la falta de un rol dedicado. El cambio hace que `porcentajeDe` trunque en vez de redondear. La descripción habla solo de descuentos, pero la función también calcula el impuesto de los pedidos.

```

reviewer_unico pide  buscar_reglas({"pregunta": "cómo se calculan los importes de dinero y descuentos en
centavos"})
reviewer_unico pide  grep_repo({"patron": "porcentajeDe\\("})
observa  [{"id": "R1", "origen": "reglas", "fuente": "CLAUDE.md", "texto": "R1. El dinero va en centavos
enteros\nTodo importe es un `number` ente
observa  6 coincidencias src/descuentos/descuentos.ts:14: ? porcentajeDe(subtotalCentavos, cupon.porcentaje)
src/pedidos/pedidos.ts:25: const impu
reviewer_unico pide  correr_checks({"tipo": "tests"})
reviewer_unico pide  leer_archivo({"ruta": "src/descuentos/descuentos.ts"})
observa  {"tipo": "tests", "codigo_salida": 0, "salida": "npm warn Unknown user config `\"auto-install-peers\"`.
This will stop working in the next
observa  src/descuentos/descuentos.ts líneas 1-39 de 39 1: import { ErrorCuponInvalido } from '../errores'; 2:
import type { Cupon } from '../tipo
reviewer_unico dice  **Resumen de la revisión** El cambio es correcto y está bien hecho: - **Correctness**:
`Math.floor` trunca hacia abajo para valo
reviewer_unico dice  {"hallazgos":[]}
[cierre] status=completed tokens={'tokens_entrada': 11381, 'tokens_salida': 483}

```

El agente tuvo delante la evidencia: su búsqueda devolvió el uso en `pedidos.ts` que calcula el impuesto. Después corrió los tests, pasaron, y concluyó que el cambio «es correcto y está bien hecho». Es la falla 0.c dentro del propio agente: los tests en verde lo convencieron. En el sistema multiagente, el reviewer de impacto tiene como única tarea preguntarse qué pasa en cada consumidor, y encontró este caso en todas las corridas.

3. La revisión que se perdió sin avisar (G01, un agente con razonamiento medio, segunda corrida). Apunta al modelo y a un defecto del sistema. Es el peor resultado de todas las mediciones: 0 de 6 problemas en un PR grande que el mismo sistema resolvió completo en las otras dos corridas.

```

reviewer_unico pide  leer_archivo({"ruta": "src/carrito/carrito.ts"})
reviewer_unico pide  leer_archivo({"ruta": "src/inventario/inventario.ts"})
reviewer_unico pide  leer_archivo({"ruta": "src/inventario/inventario.test.ts"})
reviewer_unico pide  leer_archivo({"ruta": "src/pedidos/pedidos.ts"})
reviewer_unico pide  leer_archivo({"ruta": "src/descuentos/descuentos.ts"})
reviewer_unico pide  leer_archivo({"ruta": "src/utils/dinero.ts"})
reviewer_unico pide  grep_repo({"patron": "formatearCantidad"})
reviewer_unico pide  grep_repo({"patron": "stockDe|descuentoPorVolumen|participacionPorLinea|recargoPorServicio"})
observa  src/carrito/carrito.ts líneas 1-36 de 36 1: import type { LineaCarrito, Producto } from '../tipos'; 2:
import { sumarCentavos } from '../
observa  src/inventario/inventario.ts líneas 1-57 de 57 1: import { ErrorProductoDesconocido,
ErrorStockInsuficiente } from '../errores'; 2: impor
observa  src/inventario/inventario.test.ts líneas 1-61 de 61 1: import { beforeEach, describe, expect, it }
from 'vitest'; 2: import { ErrorProduc
observa  src/pedidos/pedidos.ts líneas 1-59 de 59 1: import type { Carrito } from '../carrito/carrito'; 2:
import { calcularDescuento } from '../d
observa  src/descuentos/descuentos.ts líneas 1-48 de 48 1: import { ErrorCuponInvalido } from '../errores'; 2:
import type { Cupon } from '../tipo
observa  src/utils/dinero.ts líneas 1-23 de 23 1: const CENTAVOS_POR_UNIDAD = 100; 2: const PORCENTAJE_TOTAL =
100; 3: const DIGITOS_DE_CENTAVOS =
observa  1 coincidencias src/utils/dinero.ts:21: export function formatearCantidad(cantidad: number): string {
observa  17 coincidencias src/carrito/carrito.test.ts:34:
expect(carrito.recargoPorServicioCentavos()).toBe(258); src/carrito/carrito.ts:27: recar
...

```

El agente hizo su trabajo: leyó los archivos y resumió lo que había encontrado. El fallo vino al final, al pedirle los hallazgos en la ficha fija. El modelo gastó dos veces los 8 192 tokens de su cupo de salida razonando y no devolvió nada. El sistema registró el error en la traza, pero entregó «sin hallazgos» con estado `completed`: una revisión vacía que parece una revisión limpia. Es la misma clase de falla silenciosa de la Parte 0.a, esta vez dentro del sistema ya construido, y solo se vio por tener la traza. Se corrigió en dos sitios (Parte 4): ahora esa situación deja un aviso en el informe y el estado `incompleto`, y el reintento ya no

repite la misma llamada. También muestra una ventaja del sistema multiagente que no es de calidad sino de robustez: si a uno de sus siete agentes le pasa esto, se pierde una parte de la revisión y no toda.

Parte 3 — Frenos, probados haciéndolos saltar

Cuatro frenos, todos en código y fuera del control del modelo. Se forzaron con un modelo de guion (`evaluacion/guion.py`) que se porta mal a propósito, así que no gastaron nada: `python -m evaluacion.forzar_frenos`. Hay una traza por freno en `resultados/frenos/`.

freno	dónde	cómo se forzó	qué pasó
Tope de pasos	<code>bucle.py</code>	El guion pide una búsqueda distinta en cada turno y nunca termina; límite 4	Corta al cuarto paso; cada llamada pendiente recibe su resultado y el agente entrega el hallazgo que tenía
Presupuesto de tokens	<code>traza.py</code> , antes de cada llamada	Cada turno cuesta 30 000 tokens; límite 120 000 con 35 000 de reserva	Corta a los 91 500; la reserva alcanza para extraer el hallazgo
Detector de repetición	<code>bucle.py</code>	El guion pide siempre la misma búsqueda	La tercera llamada idéntica no se ejecuta y el agente se detiene
Tiempo máximo de un check	<code>herramientas.py</code>	El comando de tests duerme 60 s; límite 2 s	Se mata el grupo de procesos; el modelo recibe el error como observación y sigue

```
=== Presupuesto de tokens (120 000, con 35 000 de reserva para el cierre)
llamadas al modelo: 4 · usos de herramienta: 3 · status: incompleto
FRENO: {"freno": "presupuesto_de_tokens", "agente": "reviewer_unico", "detalle": "gastados 91500 tokens de
85000 disponibles"}
saltó el freno: sí
respuesta entregada:
## Informe de review

> El reviewer se detuvo por presupuesto_de_tokens: la revisión no se completó.

- **[alto] src/pedidos/pedidos.ts:24** (reglas, pendiente)
El impuesto se calcula antes de restar el descuento.
`const impuestoCentavos = porcentajeDe(subtotalCentavos, IMPUESTO_PORCENTAJE);`
traza: resultados/frenos/presupuesto_de_tokens.jsonl
```

En los cuatro casos el informe sale con un aviso arriba y la corrida queda con estado `incompleto`; nunca devuelve una respuesta vacía. La reserva del presupuesto existe para eso: el trabajo normal puede gastar hasta el límite menos la reserva, y solo los pasos de cierre (extraer los hallazgos y redactar) pueden usar el resto. Sin ella, una corrida que agota el presupuesto pierde todo lo que gastó.

Qué no ven. El presupuesto se comprueba antes de cada llamada, cuando aún no se sabe cuánto costará: puede pasarse por una llamada. El detector de repetición solo ve la llamada idéntica; no ve a un agente que alterna entre dos llamadas ni al que cambia un poco los argumentos cada vez. Eso no lo invalida, porque para esos casos queda el tope de pasos. No hay freno de confirmación humana porque ninguna herramienta escribe; sería obligatorio en cuanto el sistema aplique parches (trabajo futuro).

De dónde salen los límites

Los valores por defecto se eligieron antes de medir nada, sin una referencia que los respaldara, y eso es una debilidad del trabajo: un límite debería salir de una fuente o de una medición. Después de medir se contrastaron con lo que usaron de verdad las corridas:

sistema	corridas	llamadas del reviewer más ocupado: mediana	máximo	tokens por corrida: mediana	máximo
Baseline: un agente	42	4	10	13 439	43 360
Un agente con las instrucciones de los reviewers	42	4	8	16 938	32 099
Un agente, razonamiento medio	42	6	8	31 261	63 058
Un agente, razonamiento alto	42	5	11	21 947	59 152
Multiagente (7 agentes)	42	5	10	77 961	144 129
Multiagente sin verificador	42	4	7	52 096	73 491
Multiagente sin RAG	42	5	8	77 670	123 893
Multiagente, razonamiento alto	42	5	11	97 831	162 923

límite	valor	con qué se contrasta	veredicto
Pasos por agente	12 (10 en los reviewers, 6 en el verificador)	LangGraph 1.2.12 no pone un tope útil por defecto (<code>DEFAULT_RECURSION_LIMIT</code> = 10 007 en el paquete instalado); el tope es responsabilidad de quien escribe el agente. El máximo observado está en la tabla de arriba	Razonable: deja margen sobre el peor caso sin permitir bucles largos
Tokens por corrida	400 000, con 30 000 de reserva	No se encontró una recomendación externa con un número: depende del modelo y del costo. Solo queda la medición propia	Demasiado holgado y mal planteado: es un solo límite para todos los sistemas. Debería ser por sistema, por ejemplo el doble de su peor caso
Tiempo de un check	120 s	Vitest 4.1.11 corta cada test a los 5 s por defecto. En 234 ejecuciones medidas el check tardó 0,7 s de mediana y 1,1 s de máximo	El peor de los tres: sobra por dos órdenes de magnitud. Depende del repo revisado, así que debería estar en <code>perfil.toml</code>
Repetición	más de 2 llamadas idénticas	Ninguna referencia: es criterio	Saltó en corridas reales (abajo)
Cupo de salida y tiempo por llamada	4 096 tokens y 60 s (antes 8 192 y 180 s)	Se midieron tres cupos sobre los casos difíciles (Parte 4.3)	Faltaban como frenos: se añadieron al final, al ver llamadas de más de dos minutos que no devolvían nada

Además de los forzados, estos frenos saltaron solos en las corridas medidas:

sistema	freno	agente	veces
Multiagente, razonamiento alto	repeticion	reviewer_reglas	1

Ni el presupuesto de tokens ni el tiempo máximo de un check se dispararon en una corrida medida, así que los resultados de la Parte 2 no dependen de sus valores. Sigue faltando un freno: un límite de tiempo para la corrida completa.

Parte 4 — Extensión: cuánto razona el modelo, y código más difícil

La extensión se eligió después de medir el baseline. La pregunta fue si, antes de añadir agentes, bastaba con que un solo agente razonara más. Después se añadieron dos experimentos que salieron de dudas sobre los primeros resultados: qué pasa con cambios más difíciles, y por qué algunas corridas tardaban tanto.

4.1 — El nivel de razonamiento

El modelo de la H200 acepta un parámetro `reasoning_effort` que le indica cuánto «pensar» antes de responder. Todo el taller corre con `low`. Aquí se cambia solo ese valor, con el mismo golden set.

agentes	razonamiento	encontrados, 14 casos	hallazgos en casos limpios	tokens de entrada	segundos	encontrados, casos difíciles (4.2)
uno	bajo	12,3 (11–13) de 14	0	191 346	112	8,7 (4–11) de 14
uno	medio	14 de 14	0	380 891	530	11,3 (8–13) de 14
uno	alto	13,7 (13–14) de 14	0	296 246	194	11,0 (9–13) de 14
siete	bajo	14 de 14	0,7 (0–1)	1 022 382	351	13,7 (13–14) de 14
siete	alto	14 de 14	1,7 (0–3)	1 267 720	582	14 de 14

Con razonamiento medio, un solo agente encontró los 14 problemas en sus tres corridas, sin marcar nada en los casos limpios, con 2,0 veces los tokens del baseline; el sistema multiagente necesita 5,3 veces. A cambio es el más lento de todos: sus 14 casos suman 530 segundos frente a 351 del multiagente, porque el tiempo depende de cuánto texto genera el modelo y razonando escribe mucho más. El nivel alto no mejora al medio y añade hallazgos no esperados. En el multiagente, subir el razonamiento no encuentra más y sí marca más cosas en los casos limpios: cada reviewer tiene una tarea pequeña y no necesita pensar tanto. El nivel medio se había descartado al principio por una sola llamada de prueba; medirlo cambió la conclusión.

4.2 — Cambios más difíciles

Los 14 casos son cambios de pocas líneas. Para ver qué pasa cuando el cambio es más grande o tiene más dependencias entre sus piezas se construyeron tres casos más, sobre la misma mini tienda. Van en un conjunto aparte (`golden_set_grande.json`) para no alterar el examen de 14.

caso	qué es	problemas	líneas añadidas
G01	PR grande: cinco de los cambios pequeños (C03, C06, C08, C09, C10) juntos en un solo parche	6	74
G02	PR grande: otros cuatro (C04, C05, C07, C11) juntos	5	56
G03	Caso complejo: un módulo nuevo de precios con una jerarquía de clases (método plantilla y subclases), un contenedor que inyecta las dependencias y una función con caché	3	213

En G03 los tres bugs no están en una línea que se lea sola, sino en cómo se conectan las piezas: una subclase que sobrescribe el método plantilla y se salta el tope que la clase base garantiza; una dependencia cuyo valor se lee una sola vez al armar el contenedor, cuando debía consultarse en cada uso; y una caché cuya clave ignora uno de los dos argumentos. Con el parche aplicado, lint, tipos y tests pasan; tres tests ocultos demuestran cada bug.

sistema	corridas	problemas encontrados	regla correcta citada	hallazgos no esperados
Baseline: un agente	3	8,7 (4–11) de 14	2,7 (2–3) de 9	1,0 (0–2)
Un agente con las instrucciones de los reviewers	3	11,3 (11–12) de 14	4,3 (4–5) de 9	2,0 (1–3)
Un agente, razonamiento medio	3	11,3 (8–13) de 14	4,3 (3–5) de 9	0,7 (0–1)
Un agente, razonamiento alto	3	11,0 (9–13) de 14	3 de 9	1,0 (0–2)
Multiagente (7 agentes)	3	13,7 (13–14) de 14	7,3 (7–8) de 9	0,7 (0–2)
Multiagente sin verificador	3	13,3 (12–14) de 14	6,0 (4–7) de 9	0,7 (0–2)
Multiagente sin RAG	3	13 de 14	0 de 9	0,3 (0–1)
Multiagente, razonamiento alto	3	14 de 14	8,3 (7–9) de 9	2,3 (2–3)

sistema	corridas incompletas	llamadas al modelo	tokens de entrada	frente al baseline	tokens de salida	segundos (suma de los casos)
Baseline: un agente	0	13	77 208	1,0×	6 136	46
Un agente con las instrucciones de los reviewers	0	13	85 627	1,1×	6 417	46
Un agente, razonamiento medio	0	15	126 759	1,6×	46 643	268
Un agente, razonamiento alto	0	15	99 405	1,3×	16 978	116
Multiagente (7 agentes)	0	159	653 249	8,5×	71 413	355
Multiagente sin verificador	0,3 (0–1)	58	287 929	3,7×	49 868	261
Multiagente sin RAG	0	159	625 234	8,1×	38 115	168
Multiagente, razonamiento alto	0,3 (0–1)	183	859 870	11,1×	116 484	633

El caso complejo, bug por bug (en cuántas de las tres corridas lo encontró cada sistema):

sistema	herencia: se salta el método plantilla	inyección: valor leído una sola vez	caché: clave incompleta
Baseline: un agente	0 de 3	2 de 3	2 de 3
Un agente con las instrucciones de los reviewers	0 de 3	3 de 3	3 de 3
Un agente, razonamiento medio	3 de 3	3 de 3	3 de 3
Un agente, razonamiento alto	0 de 3	2 de 3	2 de 3
Multiagente (7 agentes)	3 de 3	3 de 3	3 de 3
Multiagente sin verificador	3 de 3	3 de 3	3 de 3
Multiagente sin RAG	3 de 3	3 de 3	3 de 3
Multiagente, razonamiento alto	3 de 3	3 de 3	3 de 3

Qué muestra. Aquí la distancia sí es grande: el baseline encuentra 8,7 de 14 y el sistema multiagente 13,7. La diferencia está sobre todo en el caso complejo: ningún agente único con razonamiento bajo o alto encontró el bug de herencia en ninguna corrida, y todas las variantes multiagente encontraron los tres bugs las tres veces. Juntar muchos cambios simples (G01 y G02) afecta menos a un agente único que la indirección del código (G03).

Pero el razonamiento medio vuelve a cerrar la distancia. Un solo agente con razonamiento medio también encontró los tres bugs de G03 en sus tres corridas, con 126 759 tokens frente a 653 249. Su promedio (11,3 de 14) queda por debajo del multiagente por una sola corrida en la que perdió todos los hallazgos de G01, que es el tercer caso de 2.c.

Lo que se puede decir. La ventaja del sistema multiagente crece cuando el problema no está en la línea que cambia sino en cómo se conecta con otras piezas, y además aguanta mejor un fallo aislado del modelo. Pero un agente único con razonamiento medio alcanza casi la misma calidad con una quinta parte de los tokens. Que repartir el trabajo ayude más cuanto más complejo es el código es compatible con estos datos, no queda demostrado: son tres casos escritos para el taller, y G03 añade archivos completos, que es menos habitual en un PR que modificar funciones.

4.3 — El tiempo, y las llamadas que no devuelven nada

El sistema multiagente no solo gasta más tokens: tarda bastante más, y en los casos difíciles la diferencia crece.

caso	baseline, segundos	multiagente, segundos	multiagente, peor corrida
Un caso de los 14 (promedio)	8	25	—
G01	19	176	342
G02	12	52	58
G03	16	127	141

Los cambios no son grandes (como mucho 213 líneas) y un agente no lee todo el repositorio: recibe el diff y abre dos o tres archivos. Al mirar las trazas llamada por llamada, la causa principal resultó ser otra: llamadas en las que el modelo gasta todo su cupo de salida razonando y no devuelve nada. Es el mismo comportamiento que la Parte 0 ya había mostrado, y reaparece con entradas más largas.

corridas (suma de las tres)	llamadas al modelo	vacías	segundos en llamadas vacías	parte del tiempo de modelo	llamada más lenta, s
Baseline, 14 casos	183	1	12	4 %	14
Multiagente, 14 casos	1287	9	22	1 %	13
Baseline, casos difíciles	38	4	12	9 %	7
Un agente con razonamiento medio, casos difíciles	45	3	118	15 %	44
Multiagente, casos difíciles	478	20	843	41 %	149
Multiagente con razonamiento alto, casos difíciles	550	24	1608	44 %	161

En los casos pequeños casi no ocurre. En los difíciles se lleva más del 40 % del tiempo, y el sistema multiagente está más expuesto simplemente porque hace unas diez veces más llamadas. Detrás había tres decisiones más que resultaron equivocadas: el cupo de salida se había subido a 8 192 tokens cuando la primera llamada salió vacía, lo que solo hacía que el fallo tardara más; el reintento repetía la misma llamada sin cambiar nada; y no había ningún límite de tiempo que cortara una llamada de dos minutos y medio.

Se corrigió en un commit aparte y se volvió a medir el sistema multiagente sobre los tres casos difíciles: cupo de salida más bajo y configurable por rol, 60 segundos como máximo por llamada, y un reintento que usa siempre el nivel de razonamiento más bajo.

versión	cupo de salida	encontrados de 14, por corrida	segundos por corrida	llamada más lenta, s	llamadas vacías
Como se midió todo lo anterior	8 192	14, 14, 13	221 a 532	149	20
Con el arreglo	2 048	13, 13, 13	220 a 270	60	30
Con el arreglo y cupo intermedio (valor que se deja)	4 096	13, 12, 14	236 a 248	60	26

El arreglo resuelve solo una parte. Acota el peor caso (ninguna llamada pasa de 60 segundos y el tiempo por corrida es más estable), pero no ataca la causa: las llamadas vacías siguen ocurriendo con la misma frecuencia, y un cupo demasiado justo hace que algún reviewer se quede sin espacio. Al probarlo se vio además que la opción que debía «apagar» el razonamiento en este servidor no lo apaga: el único ajuste que lo acota de verdad es el nivel bajo, que ya estaba puesto. Los demás sistemas no se volvieron a medir con este cambio.

Conclusiones

- Sobre cambios pequeños, varios agentes no encuentran mucho más que uno.** La diferencia fue de uno o dos casos sobre 14, del tamaño del ruido entre dos corridas, y desaparece si el agente único razona un poco más.

2. **Sobre código con más indirección, sí.** Con razonamiento bajo o alto, ningún agente único encontró el bug de herencia del caso complejo, y el sistema multiagente lo encontró siempre. Con razonamiento medio, un solo agente también lo encontró.
3. **Antes de añadir agentes conviene ajustar el que ya hay.** Mejores instrucciones y un nivel medio de razonamiento llevaron a un solo agente a resultados casi iguales con menos de la mitad de los tokens, aunque no más rápido: razonar más también tarda más.
4. **Lo que el sistema multiagente aporta con claridad** es citar mejor la norma incumplida y no perder toda la revisión cuando una llamada falla. Lo paga con cinco veces más tokens, más tiempo y algún hallazgo de más en cambios correctos.
5. **De las piezas del sistema, el RAG se justificó y el verificador no.** El RAG permite citar la regla y reduce hallazgos de más; el verificador comprueba que una afirmación sea cierta, no que sea un problema, y cuesta un tercio de los tokens.
6. **Lo que más protegió fue lo que no depende del modelo:** las comprobaciones en código, las trazas y volver a medir con un control. Varias conclusiones intermedias resultaron no ser ciertas al comprobarlas así.

Parte 5 — Reflexión

1. **¿En qué es un agente basado en objetivos y en qué no?** Lo es en que persigue un estado final (un review entregado) eligiendo acciones según lo que observa: nadie le dice qué archivo leer ni qué buscar. No lo es en que el objetivo no está representado en ningún sitio que el programa pueda comprobar. Vive como texto en el prompt (`agentes/prompts.py`) y no existe una prueba de meta: dentro del bucle, quien decide que se cumplió es el modelo, al dejar de pedir herramientas (la función siguiente de `agentes/bucle.py`), o un freno que lo corta. Tampoco planifica: decide un paso cada vez. Lo que sí decide el código es qué se publica: `comprobar_en_codigo` y el veredicto del verificador (`agentes/verificador.py`) y la comprobación de procedencia del informe (`agentes/sintetizador.py`). El objetivo lo fija una persona, el fin lo declara el modelo y la publicación la autoriza el código.

2. **Con resultados/ delante.** Sobre los mismos 14 casos, el baseline usó en promedio 191 346 tokens de entrada en 61 llamadas (3 137 por llamada) y el sistema multiagente 1 022 382 en 429 (2 383 por llamada): 5,3 veces más tokens con 7,0 veces más llamadas. Si duplicara el tope de pasos de 12 a 24, el peor caso de tokens de entrada no se duplicaría: crecería hacia el cuádruple, porque cada paso reenvía el historial entero y lo que se suma es $1 + 2 + \dots + n$ observaciones. En la práctica no cambiaría nada, por dos razones que salen de mis cifras: el agente más ocupado del baseline hizo como mucho 10 llamadas contando la de cierre, por debajo del tope, y el presupuesto de tokens es un freno aparte que no se mueve al mover el de pasos. Lo que cambiaría es cuál de los dos salta primero en una corrida descontrolada.

3. **Un despliegue que hace daño.** El paso siguiente de este trabajo es un agente que aplica parches y hace commits en el repositorio del trabajo. Ahí el daño es concreto: código incorrecto que llega a una rama compartida, o un PR ajeno con un test malicioso que `correr_checks` ejecuta. En el servidor pondría lo que aquí protegió de verdad: los comandos en una lista cerrada, el proceso con el entorno vacío y sin red, la escritura confinada a una copia, y las comprobaciones en código antes de cualquier modelo. A una persona le dejaría aprobar el plan y cada commit, como pausa del grafo y no como instrucción. Lo que solo parecía proteger lo mostró la Parte 0: los tests en verde (0.c), la instrucción «no inventes» en el prompt (0.a) y, en el caso adversarial, que el modelo ignorara el comentario que pedía no reportar: lo ignoró, pero eso fue decisión suya, no una garantía.

Cómo evolucionó el trabajo

El resultado final no salió de un solo intento. Se siguió la regla del curso, baseline → medir → extender → medir, y casi cada medición obligó a cambiar algo: el código, el examen o la forma de medir. La tabla recorre ese camino en orden; cada fila corresponde a uno o más commits del repositorio.

paso	qué se hizo	qué mostró la medición	qué se cambió por eso
1	Parte 0, antes de construir nada	El modelo de la H200 gastó los 4 096 tokens en razonar y devolvió una respuesta vacía, sin error	El nivel de razonamiento pasó a ser un ajuste del <code>.env</code> y toda llamada detecta la respuesta vacía y reintenta (<code>revisor/llamada.py</code>)
2	Primer diseño: cuatro dimensiones (bugs, reglas, código muerto, impacto) y un RAG solo con las 12 reglas del repo	Antes de medir, al revisar el diseño: un review real también mira clean code y eficiencia, y con 15 fragmentos el RAG casi sobraba (las reglas caben en el prompt)	Cinco dimensiones; reglas R13 a R16 con su fuente; el índice pasó de 15 a 206 fragmentos con documentación descargada
3	Baseline de un agente y golden set de 14 casos	11 de 14 (commit <code>dbe8dc9</code>). Las trazas mostraron los tres fallos: vio con grep que una función no se usaba y no lo reportó (C09); no miró qué hacía la llamada dentro del bucle (C10); los tests pasaron y dio el cambio por bueno (C11)	Se escribieron los cinco reviewers especializados, con instrucciones que atacan esos tres fallos. Fue un error de método, que se corrige en el paso 6
4	Capa multiagente: cinco reviewers, verificador y sintetizador	11 de 14 en la primera corrida (no se commiteó). Las trazas mostraron que el paso de unir fusionaba líneas vecinas y dimensiones distintas, y perdía hallazgos reales	Unir pasó a exigir mismo archivo, línea y dimensión. Segunda corrida: 14 de 14 (commit <code>7c85395</code>)
5	Ablaciones: sin verificador y sin RAG	Sin verificador, mismo resultado con menos tokens. Sin RAG: 13 de 14 y 6 corridas incompletas	Al preguntarse si estaba bien no haber adaptado los prompts: no lo estaba. Se había quitado la herramienta pero los prompts seguían pidiéndola. Corregido, sin RAG da 14 de 14 y ninguna incompleta: la conclusión anterior era un efecto del prompt
6	Revisión del método, buscando más errores como el anterior	Cuatro: instrucciones de los reviewers sacadas del examen, extensión que comparaba dos cambios a la vez, trazas sin contenido, y sistemas medidos con versiones distintas del código	Control con un agente único que recibe esas mismas instrucciones; multiagente con razonamiento alto; trazas con lo que el modelo pidió, observó y respondió; una medición final de todo con un mismo commit
7	Medición final de los siete sistemas	El mismo baseline que había dado 11 dio 13 de 14: la diferencia entre dos corridas era tan grande como las diferencias entre sistemas que se estaban interpretando	Tres repeticiones por sistema; las tablas muestran promedio y rango
8	Pregunta que quedaba: ¿y con código más difícil?	El repo de prueba es pequeño y un solo agente lo lee casi entero	Dos «PR grandes» que juntan varios casos y un caso complejo con herencia, inyección de dependencias y caché (Parte 4.2)
9	Medición de los casos difíciles	El sistema multiagente sí se separa del baseline (13,7 frente a 8,7 de 14). Pero tarda mucho más de lo que justifica el tamaño de los cambios	Se revisaron las trazas llamada por llamada: cerca de la mitad del tiempo se iba en llamadas que no devolvían nada. Se bajó el cupo de salida, se añadió un límite de tiempo por llamada y se cambió el reintento (Parte 4.3)
10	Razonamiento medio, que se había descartado con una sola llamada de prueba	Un solo agente con razonamiento medio encontró los 14 problemas en sus tres corridas, y los tres bugs del caso complejo	La conclusión del taller pasó de «el multiagente gana en código difícil» a «primero conviene ajustar el agente único»

Dos cosas quedan de este recorrido. La primera: varias conclusiones que parecían claras en algún momento (el baseline es mucho peor, sin RAG se pierde un caso, solo el multiagente resuelve el caso complejo) resultaron ser ruido, un prompt mal adaptado o una medición que faltaba. La segunda: todas se detectaron por lo mismo, tener las trazas y volver a medir con un control.

Errores cometidos y limitaciones

Durante el taller se cometieron varios errores de método. Se detectaron y corrigieron antes de la medición final, y quedan en el historial de commits. Se listan porque cambian cuánto se puede confiar en cada resultado.

error	qué consecuencia tenía	qué se hizo
Las instrucciones de los cinco reviewers se escribieron después de ver en qué fallaba el baseline, con pistas a la medida de esos fallos	El sistema multiagente tenía mejores instrucciones además de más agentes: no se podía saber cuál de las dos cosas mejoraba el resultado	Se añadió un control: un solo agente con esas mismas instrucciones. Las pistas siguen saliendo del examen, para los dos sistemas
El paso de unir hallazgos fusionaba líneas vecinas y dimensiones distintas	Se perdían problemas reales: la primera medición del multiagente dio 11 de 14	Se corrigió mirando las trazas del propio golden set, y se volvió a medir
En la ablación sin RAG se quitó la herramienta pero los prompts seguían pidiendo usarla	6 de 14 corridas quedaron incompletas y se «perdía» un caso: las dos cosas eran efecto del prompt contradictorio, no de la falta de RAG	Los prompts de esa ablación ya no nombran la herramienta, y hay una prueba que lo exige
La extensión comparó un agente con razonamiento alto contra el multiagente con razonamiento bajo	Dos cambios a la vez	Se midió también el multiagente con razonamiento alto
Las trazas no guardaban qué devolvía cada herramienta ni qué respondía el modelo	El análisis de fallos no tenía evidencia	Ahora guardan un extracto de cada cosa
Cada sistema se midió al terminarlo, con el código en estados distintos, y una sola vez	Se interpretaron como diferencias entre sistemas cosas que eran ruido: el mismo baseline dio 11 y después 13 de 14	Una medición final de todos con el mismo commit, y tres repeticiones de cada uno

Limitaciones que siguen en pie.

- **El repositorio de prueba es pequeño:** unos diez archivos, que un solo agente lee casi enteros en cuatro pasos. Los tres casos difíciles de la Parte 4 son una aproximación, no un repositorio real: los escribió la misma persona que el sistema, y el caso complejo añade archivos completos en vez de modificar funciones.
- **Las falsas alarmas casi no se miden:** solo hay dos casos limpios. Los hallazgos «no esperados» de los demás casos no los juzga nadie y algunos pueden ser legítimos.
- **El acierto es por ubicación y palabras clave,** con rangos de líneas amplios en algunos casos: un hallazgo flojo en la línea correcta puede contar.
- **El golden set, el sistema y los prompts los diseñó la misma persona,** y los prompts se ajustaron con el examen a la vista. No hay casos reservados sin mirar.
- **Tres repeticiones** dan un rango, no una estimación fiable de la variación.
- **Los tiempos son aproximados:** cuatro casos corren en paralelo sobre una GPU compartida, y dependen mucho de las llamadas vacías, que no ocurren igual en cada corrida.
- **El último arreglo se midió solo en un sistema.** El cambio de cupo, tiempo y reintento de la Parte 4.3 se probó con el multiagente sobre los casos difíciles. El resto de las cifras del informe son anteriores a ese cambio.
- **Las llamadas vacías no están resueltas,** solo acotadas, y falta un límite de tiempo para la corrida completa.

Trabajo futuro

El objetivo que motivó el taller sigue siendo llevar `dev-cycle` completo a LangGraph. Lo medido aquí cambia el argumento para hacerlo. No es que más agentes encuentren siempre más: es que el flujo, las aprobaciones y los límites pasan de estar en una receta a estar en código, que el sistema no depende de un proveedor y que cada paso se puede medir. Y deja un criterio para decidir cuántos agentes usar: empezar con uno bien ajustado y añadir un rol solo donde una medición muestre que aporta, como ocurrió con el de impacto. El orden previsto:

1. **Un repositorio de prueba difícil.** Código real, o escrito a propósito con muchas dependencias entre piezas, diffs grandes y dependencias a varios saltos, con su golden set y casos reservados que no se miren al ajustar prompts. Es la condición para saber si repartir el trabajo ayuda donde un agente solo se queda corto.
2. **Resolver feedback (review-comments).** Un comentario de PR es un hallazgo que viene de fuera: reutiliza la ficha y el verificador, y añade un clasificador y un redactor de respuestas. Publicar pasa por una pausa de aprobación.

3. **El ciclo completo (dev-cycle).** Planificador, implementador y explicador, con este revisor como subgrafo. Los dos «STOP» de la skill (aprobar el plan, aprobar el commit) pasan a ser nodos `interrupt` con checkpoint: la herramienta que escribe en el repo no es alcanzable sin pasar por ellos.
4. **El repositorio real.** Un `perfil.toml` para un proyecto Angular, que es donde se quiere usar. El implementador es el rol donde más sentido tiene probar un modelo distinto, y el sistema ya lo permite.

Mejoras ya identificadas en el sistema actual: verificar los hallazgos en una sola llamada en vez de un bucle por hallazgo, o quitar el verificador; hacer una búsqueda inicial de reglas compartida por todos los reviewers, porque hoy varios preguntan lo mismo; acotar la instrucción del reviewer de clean code que produce hallazgos en cambios correctos; un presupuesto de tokens por sistema y un límite de tiempo por corrida; y el tiempo máximo de un check en el perfil del repo.

Reproducibilidad

El código medido es el del commit `6a17206` (último que toca `revisor/`). Versiones fijadas en `requirements.txt` (`langgraph==1.2.12`) y en `repo-prueba/package-lock.json`. Todas las tablas de este informe las genera `informe/generar_informe.py` leyendo `resultados/`; ninguna cifra está escrita a mano.

```
uv venv --python 3.12 .venv && uv pip install --python .venv/bin/python -r requirements.txt
(cd repo-prueba && npm ci)
cp .env.example .env                                # VPN de la USFQ conectada
.venv/bin/python -m revisor.rag.ingesta               # descarga las fuentes aprobadas del RAG
.venv/bin/python -m pytest -q                        # pruebas del sistema, sin modelo
.venv/bin/python -m evaluacion.parte0.c_tests_en_verde
.venv/bin/python -m evaluacion.evaluar --verificar    # la verdad de cada caso, ejecutada
.venv/bin/python -m evaluacion.evaluar --sistema multiagente --repeticion 1
.venv/bin/python -m evaluacion.forzar_frenos
.venv/bin/python informe/generar_informe.py
```

Ninguna credencial aparece en el código, las trazas ni el repositorio: se leen de un `.env` que `.gitignore` excluye, las herramientas no pueden leerlo y los comandos corren sin las variables del entorno. Se buscaron patrones de clave en `resultados/` antes de cada commit.